

Assignment 2(week 3 and 4)

Part 1:

Q1. The query returns 0 because in SQL, NULL is represented by an unknown value, so = does not work here, or any comparisons. Instead he/she should use IS NULL to correctly figure out the total NULLs.

SELECT COUNT(*) FROM Student WHERE phone_number IS NULL; will work.

Q2. Since the GROUP BY department is there to ensure that the department in the output will be unique, so yes we can say that the DISTINCT used here is redundant.

Q3. The difference in row count indicates that there are records in the left table that do not have any corresponding matching records in the right table. This happens when the customer has an say registered an account somewhere, but hasn't used it, for e.g. he/she has an account on a food service but hasn't placed an order yet.

Q4. Employee name is not in the GROUP BY clause, if SQL allowed this, it'll just show department and the avg salary, without the employee's name. The correction for that is: SELECT department, AVG(salary) FROM Employee GROUP BY department;

Q5. we should use HAVING instead of WHERE here as, AVG() doesn't not work for clauses like WHERE.

Correct code: SELECT department, AVG(salary) FROM Employee GROUP BY department HAVING AVG(salary) > 80000;

Q6. a) Non-correlated: Only once we need overall avg to be calculated.

b) Correlated: Since employee attribute is asked so subquery must reference outer query's dept id to calculate the avg for that employee's dept.

Performance implications: A non-correlated subquery executes just once, but a correlated subquery executes repeatedly for every single row in the outer query.

PART II:

a) SELECT c.customer_id, c.name, c.email, c.phone
FROM Customer c
LEFT JOIN `Order` o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

Justification: LEFT JOIN with IS NULL filter efficiently, since they'll be present in the left due to their registration but not in the right as they have never placed an order.

b) WITH totorder AS (
SELECT o.restaurantID, o.orderID, SUM(mi.price * oi.quantity) AS orders
FROM `Order` o
JOIN OrderItem oi ON o.orderID = oi.order_id
JOIN MenuItem mi ON oi.itemID = mi.itemID
GROUP BY o.restaurantID, o.orderID
)
SELECT restaurantID, AVG(orders) AS avgOrderVal
FROM totorder
GROUP BY restaurantID
HAVING COUNT(orderID) > 5;

Firstly find out the sum then use HAVING to filter for restaurants where count > 5.

c) The totorder part will remain the same.

We'll change:

RestAves AS (
SELECT restaurantID, AVG(orders) AS avgOrderVal
FROM totorder
GROUP BY restaurantID
HAVING COUNT(orderID) > 5
)
SELECT restaurantID
FROM RestAves
WHERE avgOrderVal > (SELECT AVG(orders) FROM totorder);

Justification: since platform wise avg is fixed, used non correlated subquery.

d) WITH ItemQuants AS (
SELECT mi.restaurantID, mi.itemID, mi.itemName, SUM(oi.quantity) as totqty
FROM MenuItem mi
JOIN OrderItem oi ON mi.itemID = oi.itemID
GROUP BY mi.restaurantID, mi.itemID, mi.itemName
)
RankedItems AS (

```

SELECT restaurantID, itemID, itemName, totqty,
       RANK() OVER(PARTITION BY restaurantID ORDER BY totqty DESC) as rank
FROM ItemQuants
)
SELECT restaurantID, itemID, itemName, totqty
FROM RankedItems
WHERE rank = 1;

```

Justification: GROUP BY + MAX() fetches the highest numeric value but loses the row-level context, hence the special RANK() function is used.

PART III:

Q1: Func dependencies:

FD1 student_id is dependent on student_name

FD2 course_id is dependent on course_name and instructor_id

FD3 instructor_id is dependent on instructor_name, instructor_office

FD4 student_id, course_id is dependent on grade, enrollment_date.

Q2. yes it is 1NF,

as the basic rule of 1NF is :

- using row to convey info is not permitted.
- no mixing of data
- table without primary key(follows with above)
- no repeating grps

Q3. partial dependency of student_id dependent on student_name and course_id dependent on course_name. Dependence is for only a part of the primary key.

Insertion anomaly while adding a newly created course if not student has enrolled in it, since student id is part of PK.

Q4. Transitive dependency: course_id is dependent on course_name and instructor_id

instructor_id is dependent on instructor_name, instructor_office.

Update Anomaly, is office is changed, have to update office in every row, so risk of inconsistent filling of data.

Q5. Student(id and name) FD1

Instructor(id, name, office) FD2

Course(id, name, instructor_id) FD3

Enrollment(student_id, course_id, grade, enrollment_date) FD4

Part IV:

a) For query (a) in Part II, finding customers with no orders can be achieved via LEFT JOIN or NOT IN subquery. I chose the LEFT JOIN due to its filtering quality for NULLs. If the subquery in a NOT IN clause returns a NULL value, the entire expression evaluates to UNKNOWN, causing the query to return 0 results unexpectedly. A LEFT JOIN checking for IS NULL avoids this logical trap and visually represents the difference cleanly.

b) In Part III, normalizing the Enrollment table into 3NF eliminates redundancies and anomalies, but trades off readability. Fetching a simple class with student names, instructor details, and grades now requires joining four distinct tables. This increases computational overload. Real-world systems, intentionally keep some denormalization (like storing the instructor's name directly in the course table) to optimize read performance and reduce expenses.